بحث تطبيقات الانترنت

كيف تتعامل المواقع مع تفاعلات المستخدم باستخدام JavaScript

اسم الطالب: يوليوس سامي ميخائيل

الرقم الجامعي: 2501633

كيف تتعامل المواقع مع تفاعلات المستخدم باستخدام JavaScript

بحث عن نظام الأحداث (Events) في المتصفح

مقدمة

عندما يفتح المستخدم موقعًا حديثًا فإنه لا يتعامل مع صفحة ثابتة يقرؤها فقط، بل مع واجهة تستجيب لما يفعله: ضغطة على زر تفتح قائمة، وكتابة في مربع البحث تُظهر اقتراحات، وتمرير للصفحة يحمّل محتوى إضافيًا. هذه الاستجابة تتم عبر لغة JavaScript التي تعمل داخل المتصفح (Browser) وتراقب ما يحدث في الصفحة ثم تنفذ كودًا مناسبًا لكل حالة.

في هذا السلوك يقوم هذا السلوك على ما يُعرف بالبرمجة الموجهة بالأحداث (Event-Driven Programming). في هذا الأسلوب لا ينفذ البرنامج تعليماته من أول سطر إلى آخر سطر ثم يتوقف، وإنما ينتظر وقوع حدث معين، وعندما يقع ينفذ الجزء المرتبط به. يهدف هذا البحث إلى شرح هذه الآلية بشكل مبسط: ما المقصود بالحدث، وكيف يربط المطور الكود به، وماذا يحدث للحدث داخل المتصفح قبل أن يصل إلى الكود، وما الأساليب المستخدمة للحفاظ على سرعة الموقع.

أولًا: مفهوم الحدث (Event)

الحدث هو إشارة يرسلها المتصفح ليخبر الصفحة بأن شيئًا ما قد حدث. مصدر هذه الإشارة قد يكون المستخدم نفسه، كالنقر بالفأرة (click) أو الضغط على مفتاح من لوحة المفاتيح (keydown) أو لمس الشاشة (touchstart) أو إرسال نموذج (submit). وقد يكون المصدر المتصفح، كانتهاء تحميل الصفحة (load) أو تغيير حجم النافذة (resize) أو انقطاع الاتصال بالإنترنت (offline). وبذلك فإن الأحداث ليست مقصورة على تصرفات المستخدم، بل تشمل كل تغير يهم الصفحة.

ثانيًا: الـ DOM وعلاقته بالأحداث

عند تحميل الصفحة يحوّل المتصفح كود HTML إلى بنية شجرية من الكائنات تسمى DOM (Document Object Model). في هذه الشجرة، يصبح كل وسم في الصفحة، من الأزرار إلى الصور والفقرات، عقدة (node) في هذه الشجرة. تقع الأحداث على هذه العقد، وتصل لغة JavaScript إليها من خلال واجهة الـ DOM، فتستطيع

على أنه حلقة الوصل بين اللغة والصفحة DOM قراءة محتواها وتغييره وتعديل شكلها. ولهذا يُنظر إلى الـ المعروضة.

ثالثًا: ربط الكود بالحدث (Event Listeners)

على العنصر Callback أو Event Handler حتى تستجيب الصفحة لحدث ما، يسجل المطور دالة تسمى المطلوب، ويطلب من المتصفح تنفيذها عند وقوع الحدث. وأفضل طريقة لذلك هي الدالة addEventListener، كما في المثال التالي:

```
const btn = document.querySelector('#menuBtn');
btn.addEventListener('click', () => menu.classList.toggle('open'));
```

أو إسناد دالة onclick، مباشرة مثل HTML هناك طريقتان قديمتان لنفس الغرض: كتابة الحدث داخل وسم الأولى تخلط بين هيكل الصفحة وسلوكها فتصعّب صيانة الكود، .element.onclick لخاصية العنصر مثل فتسمح addEventListener والثانية لا تسمح إلا بدالة واحدة لكل حدث، فأي إسناد جديد يمحو القديم. أما بتسجيل أكثر من دالة على الحدث نفسه، وتتيح التحكم في مرحلة انتشار الحدث، ويقابلها removeEventListener لإزالة المستمع عند انتهاء الحاجة إليه، وهو إجراء مهم لتفادي تسرب الذاكرة (Memory Leaks) في التطبيقات الكبيرة.

رابعًا: كائن الحدث (Event Object)

عند استدعاء الدالة يمرر المتصفح إليها تلقائيًا كائنًا يحمل معلومات عن الحدث يسمى .Event Object من التي تحدد type التي تشير إلى العنصر الذي وقع عليه الحدث فعليًا، والخاصية target أهم ما يحويه الخاصية في أحداث لوحة المفاتيح لمعرفة المفتاح المضغوط. ويحوي الكائن أيضًا دالتين key نوع الحدث، والخاصية التي تمنع السلوك الافتراضي للمتصفح، كإعادة تحميل الصفحة عند إرسال preventDefault مهمتين: التي توقف انتقال الحدث إلى العناصر الأخرى. ومن stopPropagationالنموذج أو الانتقال إلى رابط، و في المتصفح قبل إرسالها (Validation) فحص بيانات النموذج preventDefault الاستخدامات الشائعة لـ إلى الخادم (Server)، مما يوفر على المستخدم انتظار الرد ويقلل الطلبات غير الضرورية.

خامسًا: مراحل انتشار الحدث

يمر الحدث ،body داخل div الحدث لا يقع على العنصر المضغوط وحده. فلو ضغط المستخدم على زر داخل ثم document ثم window) وفيها ينزل الحدث من أعلى الشجرة Capturing بثلاث مراحل. تبدأ بمرحلة وفيها يصل الحدث إلى العنصر نفسه. Target حتى يصل إلى العنصر المستهدف. ثم مرحلة (العناصر الأبوية وفيها يصعد الحدث مرة أخرى من العنصر إلى أعلى الشجرة، فيفعّل أي مستمع Bubbling ثم مرحلة ويمكن جعلها ،Bubbling مسجل على العناصر الأبوية في طريقه. وتعمل المستمعات افتراضيًا في مرحلة كمعامل ثالث. وفهم هذا التسلسل ضروري لأنه الأساس true بتمرير القيمة Capturing تعمل في مرحلة الذي تقوم عليه تقنية تفويض الأحداث.

سادسًا: تفويض الأحداث (Event Delegation)

لنفترض قائمة تحتوي على مئات العناصر، ويحتاج كل عنصر إلى الاستجابة للنقر. تسجيل مستمع مستقل لكل عنصر يستهلك ذاكرة أكبر، كما أن العناصر التي تضاف لاحقًا إلى القائمة لن يكون لها مستمع. الحل أن يوضع مستمع واحد على العنصر الأب، ثم يُعرف العنصر الذي وقع عليه الحدث فعليًا من خلال event.target، مستفيدين من أن الحدث يصعد إلى الأب في مرحلة Bubbling. تقلل هذه الطريقة عدد المستمعات، وتعمل تلقائيًا مع العناصر المضافة ديناميكيًا، وتجعل الكود أبسط في الصيانة. وتعتمد كثير من المكتبات وأطر العمل (Frameworks) على فكرة مماثلة في نظامها الداخلي.

سابعًا: الـ Event Loop والتعامل مع الأحداث دون تعطيل الصفحة

JavaScript لغة تعمل بخيط تنفيذ واحد (Single-Threaded)، أي أنها تنفذ مهمة واحدة في اللحظة الواحدة. ومع ذلك يستطيع الموقع الاستجابة للمستخدم وتحميل بيانات من الإنترنت في الوقت نفسه، ويرجع ذلك إلى آلية الـ Event Loop. فالأوامر تُنفذ داخل ما يسمى Call Stack، أما العمليات البطيئة مثل الطلبات الشبكية (fetch) والمؤقتات (setTimeout) فيتولاها المتصفح عبر Web APIs خارج الـ Stack. وعندما تنتهي العملية تدخل دالتها المرتجعة إلى طابور الانتظار (Callback Queue)، وتراقب الـ Event Loop الـ Call Stack باستمرار، فإذا فرغ نقلت إليه أول مهمة منتظرة. وتُعطى الـ Promises أولوية أعلى عبر طابور مستقل يسمى Microtask Queue.

يفسر هذا سبب تجمد الصفحة عندما يحتوي معالج الحدث على كود ثقيل، كحلقة تكرار ضخمة، إذ يبقى الـ Call Stack مشغولًا ولا تجد الأحداث الجديدة فرصة للتنفيذ. ولذلك يُفضَّل تنفيذ العمليات الطويلة بشكل غير متزامن (Asynchronous) باستخدام async/await مع fetch، وهي الأساس الذي تقوم عليه تقنية AJAX التي جعلت الصفحات قادرة على تحديث جزء من محتواها دون إعادة تحميلها كاملة.

ثامنًا: التحكم في الأحداث كثيرة التكرار

بعض الأحداث مثل scroll وmousemove وresize وinput قد تقع عشرات أو مئات المرات في الثانية. لو نُفذ كود ثقيل مع كل مرة لتباطأ الموقع بوضوح. ولمعالجة ذلك تُستخدم تقنيتان. الأولى Debounce وفيها يؤجَّل تنفيذ الدالة إلى أن يتوقف تكرار الحدث لمدة محددة، وتناسب مربع البحث حيث لا حاجة لإرسال طلب مع كل حرف، بل بعد أن يتوقف المستخدم عن الكتابة. والثانية Throttle وفيها يُسمح بتنفيذ الدالة مرة واحدة فقط خلال فترة زمنية ثابتة مهما تكرر الحدث، وتناسب أحداث التمرير. كما يوفر المتصفح الخيار passive في addEventListener لأحداث scroll وtouch، وهو يخبر المتصفح مسبقًا بأن المعالج لن يستدعي preventDefault، فيستمر التمرير بسلاسة دون انتظار انتهاء تنفيذ الدالة.

تاسعًا: إمكانية الوصول (Accessibility)

لا يعتمد جميع المستخدمين على الفأرة، فبعضهم يستخدم لوحة المفاتيح فقط وبعضهم يستعين بقارئات الشاشة (Screen Readers). لذلك ينبغي عند بناء التفاعلات استخدام عناصر HTML المناسبة مثل button وa بدل div، لأنها تدعم لوحة المفاتيح تلقائيًا (مفتاحا Enter وSpace) ويتعرف عليها قارئ الشاشة. وإذا اضطر المطور لاستخدام عنصر آخر فعليه إضافة الخاصيتين role وtabindex وتمكين الاستجابة لحدث keydown. والقاعدة العامة أن كل تفاعل ممكن بالفأرة ينبغي أن يكون له بديل بلوحة المفاتيح.

خاتمة

يتضح مما سبق أن تفاعل المواقع مع المستخدم يقوم على نظام متكامل: يرصد المتصفح الحدث، وينشئ كائنًا يصف تفاصيله، ثم ينقله عبر شجرة الـ DOM في مرحلتي Capturing وBubbling، وينفذ المعالجات المسجلة بواسطة addEventListener دون تعطيل الواجهة. ومن الممارسات المهمة في هذا الإطار استخدام Event Loop في القوائم الكبيرة، وضبط Event Delegation بدل الطرق القديمة، والاستفادة من الأحداث كثيرة التكرار بـ Debounce وThrottle، ومراعاة Accessibility ليكون الموقع صالحًا للجميع. ويُعد فهم هذه الأسس خطوة تمهيدية قبل تعلم أطر العمل الحديثة مثل React وVue، لأنها جميعًا مبنية فوق نظام الأحداث نفسه الذي يوفره المتصفح.

المراجع

- MDN Web Docs: Introduction to events
- MDN Web Docs: EventTarget.addEventListener()
- javascript.info: Introduction to Browser Events
- MDN Web Docs: Concurrency model and the event loop
- WHATWG DOM Standard: Events